

Routing

Introduction to Routing

We talked about HTTP and various methods like GET and POST. So to go off of that I'm going to be talking about routing and routing obviously is very important and a lot of it can be abstracted through modern frameworks but before we touch any syntax or libraries I really want to explain what routing is.

So routing is not just like an Express or FastAPI or Next.js thing. Routing exists in every backend system that's ever been built and its entire job is to answer one question: Given an incoming request, which piece of code should handle that request?

Suppose that we have a client and we have a server. When a request hits your server, it doesn't automatically know what to do. It doesn't know if this is a login attempt, if it's a data fetch, or if it's some sort of payment. Something has to decide what the server does when it receives the request. This is routing.

Routing Is Not Just URLs

There's a common misconception that URLs are routing, but that's not the case. Most beginners think about routing as URLs, but that's not true. URLs are just signals. Routing is about interpreting those signals.

You can think of the routing layer as like a switchboard operator. You have a call that comes in and then the operator listens to who the call is for and then they forward it to the right department. The router doesn't actually solve the problem. They just connect the call.

So in essence, routing doesn't actually do work. It simply decides where the work should happen. And if your routing is complicated, then your system is already in trouble.

Most every confusing back-end bug that I've ever seen comes from bad routing decisions or business logic that's leaking into the routing layer. And once these two mix, everything can feel random. A good technically sound routing layer will leave you with clarity. And if you have a bad routing layer, it's just going to be chaos on the back end.

Routing as a Pure Decision Layer

So here's the rule that I want us to follow for the rest of this video. Routing is a pure **decision layer**. No business logic, no side effects, just: does this request match this rule.

Routing can **only decide based on information that it's received**.

So now that we know that routing is just a decision layer, the next obvious question is what information does it use to make that decision? Because routing can't read the mind of the client, it can only work with the request that it's given.

What Information Does Routing Use?

If we have a box that represents an HTTP request and inside it, maybe it's got a **method**, it's got a **path**, it has **headers**, maybe some **query parameters** and a **body**. This is what your server actually receives.

Like we talked about in the last video, a request is a structured data object that shows up to your server. And routing is basically just pattern matching against that object.

Routing is selective and so not every part of the request matters for routing, but it **mainly cares about two things**. It cares about the **method** and it cares about the **path**. These two drive almost every routing decision you'll ever make.

```
GET /users
POST /users
DELETE /users/42
```

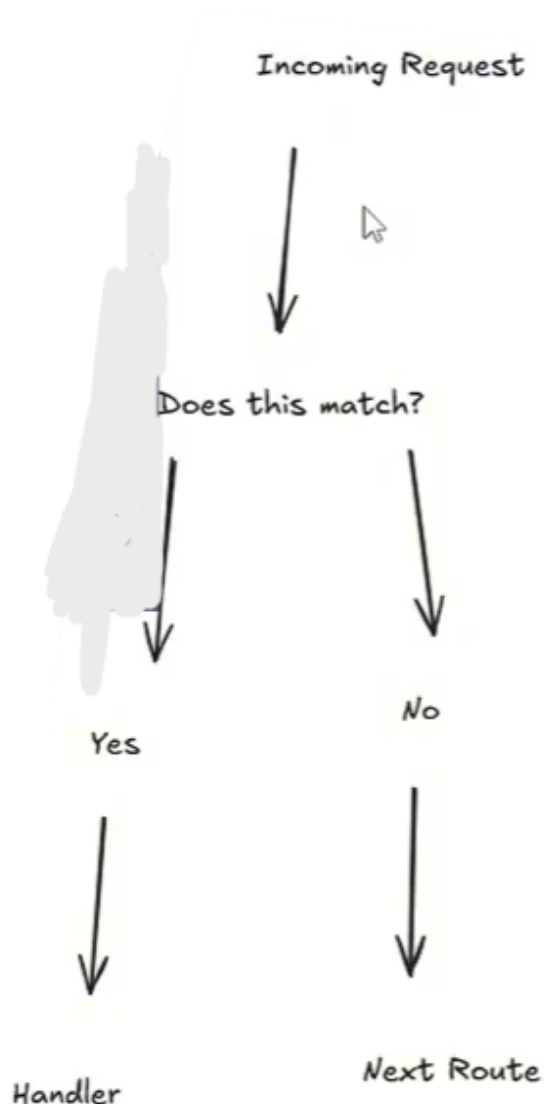


Obviously, the HTTP method tells the server what kind of action the client wants, but these are not interchangeable. The router treats these as completely separate things.

```
/users
/users/42
/orders/9182
```

The path tells the server or the router what resource the client is talking about, and these are usually nouns. Think of these as nouns. If we have some example paths right here, these paths simply just identify things and the method tells you what to do with the thing.

Routing doesn't decide based on meaning. It decides on matching patterns.



This is essentially at a high level the entire routing engine regardless of framework. Every router works the same. It'll read the request. It'll compare the method and the path and pick the first matching rule.

Request Intent

METHOD + PATH = INTENT

So now we know that routing is just a decision layer and requests are structured input and method and path are the primary signals for that input.

At this point we know two things. Routing makes the decisions and requests carry that information for the decisions. Now we're going to connect the two.

When a request hits your server it's not asking a question. It's saying to the server here's what I want you to do. Routing exists to interpret the intent.

GET /users/42 Retrieve user with ID 42	POST /users Create a new user
---	--------------------------------------

Suppose we have a GET method on the path users and then user 42. In plain English, you're just telling the server to retrieve a user with ID 42. No ambiguity whatsoever. The request has a very specific meaning.

If you have a POST at the user's route, simply create a new user. Same resource here, but completely different intent.

As I said, the path tells you what and the method tells you what to do. Together, they form the intent.

Why Methods Matter

So why do methods even matter?

Take this for example. We have three paths. GET user, create user, and delete user. This is an anti-pattern in modern routing. This is routing by verbs. And this is a very front-end mindset.

GET /users/:id
POST /users
DELETE /users/:id

Instead, what I like to see is this. We have a clear separation of concerns, all following the same path noun, but performing different actions.

And clean routing honestly isn't about being clever. It's just about having predictability. If someone can look at your route and immediately understand the intent, then you've done your job correctly.

Route Matching and Order

```
GET /users/me
GET /users/:id
```

But this can also be where routing bugs start to come into play. This is when two routes express similar intent, but your system can't tell the difference. Both of these look valid, but one is more specific.

So now we have intent, but there's a problem. What happens when multiple routes could match the same request? That's where route matching and order come in.

But honestly, before we get into that, like we really need to demystify it.

At this point, you should already feel that routes are a little less mysterious. But I'm going to completely remove all of the ambiguity so that you understand exactly what a route actually is.

What Is a Route?

Because at the end of the day, like a route is not just a function. It's not business logic and it's not even really like code. Route is literally just a rule that says: if a request looks like this, run that code.

Literally every route in a backend system can be reduced to pretty much this:

```
IF:
  Method = GET
  Path   = /users/:id

THEN:
  run getUserId()
```

If the method is this and the path is this, then run the appropriate response.

So if we have a GET method and the path is the user's path and we have a specific ID to get a user, then naturally we need to get the user by ID.

And when you see this part, this ID with the colon, it's not some magical thing. It's just a placeholder. Basically what it's saying is I don't care what the value is as long as it exists here.

So that way we can access different users in the routes like 42, 9000, you know, we can get any user as long as they exist and naturally all of these match the same route.

Matching Does Not Equal Logic

But an important distinction to be made is that matching does not equal logic.

Routing never really asks the questions like is this user allowed to do something or does this record even exist. That's not the job of the router. The validation happens after we have a route match and we'll get into that later.

And naturally, oftentimes we don't just have one route. We have a route table that can be sent through multiple sub routes. You have a list of them and the router just checks them one by one.

Routes are evaluated in order

First match always wins

This is just a rule book for your router and I want this to be crystal clear. **Routes are evaluated in order and the first match always wins.**

This is a very simple approach and that simplicity is intentional by the router. It's why routing is able to scale. You can reason about this, you can debug it and you can predict it when you have this simplicity.

If your router ever hits the wrong code, then one of two things can happen. Either A, the route matched when it shouldn't have or B, a more specific route came along later.

Route Matching Order

Now that we understand this fundamental idea that routes are rules, we can talk about the dangerous part. What happens when multiple rules could match the same request?

And that's where route matching order becomes critical.

I want to make it painfully clear that routing is both deterministic and order dependent. This section is going to explain 90% of routing bugs. And if you've ever thought to yourself, why is the wrong route firing? This is why.

The First Match Wins

When a request hits your server, routing doesn't scan the whole list and pick the best option. It checks the routes top to bottom. And the moment that it finds a match, it stops.

The first match wins.

Like I said, it can really be broken down like this. We have an incoming request. It'll check route one. If it's a match, then it'll run that handler. If it's not, it'll check route two. And then so on and so forth.

There is no backtracking in routing. The routing only takes one pass.

Specific Routes Before General Routes

Now let's take a look at something that looks fine but isn't really under the hood.

Imagine we have two routes and then this request comes in. You can think of me here as a value that points to a user ID.

1. GET /users/:id

2. GET /users/me X

So it scans the list of routes and route one says does this match GET the path of users by ID and yes it does. So the server stops and route 2 never runs.

This isn't a bug with Express or FastAPI. This is your fault for implementing it this way. Just saying it as it is.

1. GET /users/me
2. GET /users/:id

Now, the correct routing order would actually look something like this in your code. Now, the intent is completely unambiguous.

The reason for that is because this route (1) is much more specific than this (2). There's no ambiguity here. This me route points to a specific user whereas this (2) is a more general representation of it could be any user, any ID.

<i>Pattern Matching > Meaning</i>	<i>Greedy parameters</i> <i>wildcards</i>  <i>Overlapping routes</i> <i>Poor naming</i>
--------------------------------------	---

I want to emphasize this: pattern matching is much more important than meaning. These are the kinds of patterns that lead to greedy parameters, wild cards, overlapping routes and poor naming.

Wildcards and Greedy Parameters

GET /files/*

GET /files/config

This is an example of a wild card. This route (top one) if above this will steal everything if it comes first because this is more ambiguous than this (last route).

The more specific route being below the less specific route.

Path Parameters vs. Query Parameters

Next topic I'm going to discuss is path parameters versus query parameters.

At some point you're going to ask should this be a path parameter or a query parameter? And if you don't have a rule you'll guess. And guessing leads to inconsistency with your APIs.

A path parameter is simply a parameter to identify **what resource you're talking about**. Whereas a query parameter modifies **how you actually want it**.

We have **identity** v/s **behavior**.

Identity

Behavior

/orders/9182
/posts/abc123

resource

/users?sort=asc
/users?page=2
/users?limit=20

representation

Path Parameters: Identity

Path parameters answer the question of which thing do we want. These values that are underlined identify a specific resource within a path. Without them, the request wouldn't make sense.

Query Parameters: Behavior

Query parameters on the other hand identify of course how we want the data? Do we want it sorted in a specific order? Do we want a whole page? Do we want to just get some users up to a limit?

These don't change what resource you're fetching. They just change how the server responds to the request and what data it gives you.

To put it very simply, you can think of this (Identity) as the resource and this (Behavior) as the representation of that resource.

I want you to look at both of these requests. They both target the same path and the same user, but they're very distinct.

resource /users/42
representation /users/42?include=posts

You can think of this (1st) as the specific resource and this (2nd) as the representation of that resource.

Common Path and Query Parameter Anti-Patterns

So what are some common anti-patterns with this?

/getUser?id=42
/deleteUser?id=42

This is not solid whatsoever. Like I said, we don't want verb paths. These are describing behavior and your query is describing intent and that has it backwards.

A very important **design choice when designing routes is if something is required to identify the resource, it belongs in the path. Whereas if it's optional or adjustable, it belongs in the query.**

Combined Example

So here's a combined example.

GET / users/42?fields=name,email

Running a GET method to acquire some data. We're calling the user's path. This (42) is our specific user ID, what user we want to get the data from. And then this field simply states what data we want.

So we say we want the name and we want their email.

User 42 is the resource. And the field just shapes the response.

Routing vs. Business Logic

So now we can talk about routing versus business logic.

And business logic, simply put, is just the action that your app is going to take on the back end, whether it's saving data, uploading something, etc.

This is where a lot of backends actually go wrong. Not because the developer is necessarily bad, but because they're just letting routing have too much power, letting it do too much.

As I've stated, routing is not where the work should happen at all. Routing decides where a request goes, and the business logic decides what actually happens.

If those mix, that's when you get fragile systems.

Where Does Not Equal What

A point I want to hammer home. Where does not equal what?

WHERE != WHAT

Please keep this in your mind when you're thinking about routing.

Where does not equal what?

What Routes Should and Should Not Do

So, we'll talk about what routes should and should not do explicitly.

Routes should match, extract, and forward requests, but they should not talk to the database, apply any business logic, make decisions about permissions, or transform complex data.

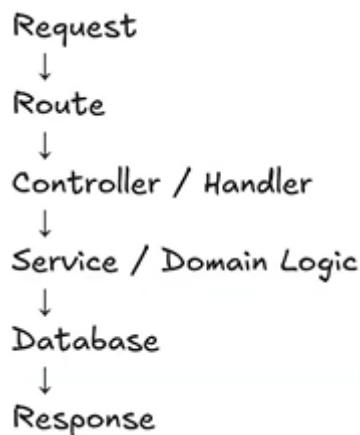
~~DB queries
Auth rules
Calculations
Side effects~~

These are the only things your route should ever be doing.

All of these things big no-no in routing.

The Request-Response Loop

So this is a more developed overview of the request response loop.



You have a request and it gets matched to a certain route. That route then sends it to a controller or some sort of handler. That controller sends it to the right service or business logic code.

It talks to the database, it does what it needs to do on the back end, and then finally we receive the response.

When you have logic that leaks into your route, it's really painful because testing becomes 10 times more difficult. Reuse becomes almost impossible. And changes will ripple throughout your entire backend architecture and instead of actually building a system, you're kind of just duct taping things together and hoping it works.

```
app.get('/users/:id', async (req, res) => {  
  // auth  
  // db query  
  // business rules  
  // response shaping  
})
```

This may just look like a normal GET request, but this is a trap. Like I said, if you're doing any sort of off querying, business logic, response shaping, it doesn't belong in this router whatsoever.

GET /users/:id
→ getUserController

Controller → Service → DB

This follows a much better structure with a clean separation of concerns. Now each part is testable.

If we have all of these separate things inside of the route, then it's hard to pinpoint any specific failure if something were to fail. Whereas down here, we can test each individual part because we're separated from the concerns.

Routing in Frameworks

So now I want to talk about why frameworks feel different from standard vanilla routing, but they really aren't.

And you might think that you don't understand routing, but oftentimes it's probably just that you don't understand a specific framework. And that comes mostly from, I believe, a lack of fundamental knowledge.

Every backend framework does the same three things when it comes to routing.

1. First, it stores a list of routes.
2. Second, it matches incoming requests against them.
3. Third, it calls the handler for the first match.

Routing Engine

Routes List
↓
Matcher
↓
Handler

That's it. This is literally your entire routing engine.

Different frameworks, you know, do different things. Express exposes this engine directly. FastAPI adds validation and typing but the core idea is identical.

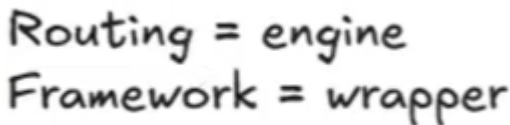
Express → manual
FastAPI → typed
Rails → convention

Different frameworks might have different mapping but they all work the same under the hood. They don't necessarily change how routing works. They just change what kind of work and how much work you have to do.

Why This Matters

So why does this matter? Why does this matter long term?

Well, I would say if you're learning routing through a framework like Express or FastAPI or Rails, you'll always feel locked into that framework. But if you learn routing conceptually, frameworks become interchangeable tools and you're much more flexible as a back-end developer.

	The mental model I think of is this. Routing is simply the engine or the mechanism and the framework is the wrapper for that engine. Always learn the engine.
---	--

This will also lead you to be able to debug quicker because you're not going to ask questions like what does Express do or what did Express do?

You'll think: which route is matching and why?

Three Questions for Debugging Routing

So these three questions right here:

- What route exists?
- What order are they in?
- What does the request look like?

If you can answer these three questions, then you can debug any routing issue.

Common Routing Mistakes

I've pretty much exhausted the core philosophy behind routes. So let's discuss, now, some common routing mistakes and hopefully you can learn from this and not go down some of the same paths that I did when I first started doing backend.

Mistake 1: Overloading Routes

First mistake is overloading routes. One route trying to do too much work on its own.

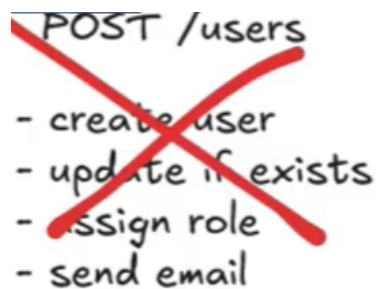


```
graph TD
    A[POST /users] --> B{ }
    B --> C[- create user]
    B --> D[- update if exists]
    B --> E[- assign role]
    B --> F[- send email]
```

This is an example of an overloaded route. This is just a decision tree at the end of the day.

We have this route POST at users and we're using this to create a user, update if it's existed, assign roles, send emails.

It's just too bloated.



```
POST /users
- create user
- update if exists
- assign role
- send email
```

The fix to this is to introduce multiple routes.

```
POST /users
PATCH /users/:id
POST /users/:id/roles
```

So now we have our initial POST routes at the user path, but we have a PATCH at users of ID so that we can update the users separately.

And then also we have a POST at a subpath of users of a specific ID of a certain role.

One intent per route.

Mistake 2: Using Verbs in Paths

If your path contains verbs, stop.

It's dogma. It's not good. Don't put verbs in your paths. Only nouns, please.

So a bad example would be something like this. We have a path `/createUser`, `/updateUser`, `/deleteUser`. This is dogma.

Instead, we have specific methods at the existing routes.

```
POST /users
PATCH /users/:id
POST /users/:id/roles
```

So if we want to create a user, we just have a POST at the base user's path.

If we want to update a user, we just PATCH at the user's path of a specific ID.

And then obviously if you want to delete a user, call the DELETE method on the user's path of a specific ID.

Mistake 3: Having Logic in Routes

The third mistake is having logic in your routes.

This kills maintainability. It absolutely just murders it.

You have something like this inside your route. Dogma. Every route becomes like a brittle snowflake when you do this.

It's just like it's too verbose. You're making the route do too much.

You need to separate the concerns.

Instead, it should look something like this. You centralize your logic and the routes stay thin.

Clean separation of concerns and you can cleanly test and debug issues.

Mistake 4: Inconsistent Naming

The next mistake I often see is inconsistent naming.

This here is too broad. There's too many options.



Just for each specific thing, pick one noun and stick with it and use that same path.

At this point, routing should be obvious. I hope if I've explained it good enough.

Final Routing Mental Model

So, let's compress everything that I've said over the course of this video into one model.

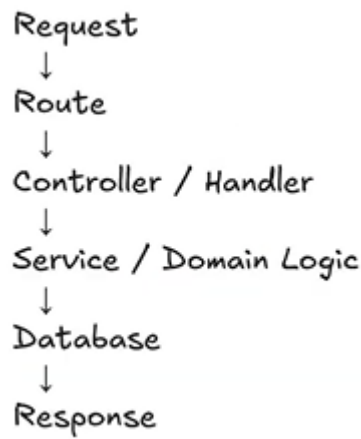
Routing is intent -> behavior mapping	Routing is the process of mapping request intent to code behavior.
---------------------------------------	--

Your request intent should be mapped to a specific behavior.

Request intent → code behavior Intent —————> Behavior	There's two halves here, the routing and the logic. Routing stops at the boundary.
LEFT (Routing) ----- Method Path Match RIGHT (Logic) ----- Controller Service Rules	We have the method, the path, and the match. Like I spoke about, the logic handles the controllers, the services, and the rules behind your actual code execution.

The Request Journey

Every request follows the same journey.



You have a request, it gets matched to a route, it goes to a handler, it is sent to some business logic, and then finally once that runs, you receive back a response.

Nothing extra, nothing hidden.

And when something breaks, you don't need to panic. You just need to follow this chain because now that everything is separated, you can cleanly walk down the line and find where the specific break point is.

Final Takeaway

If routing feels complex to you, then your system design is probably the problem, not your intelligence or anything like that.

Simplify the routes and the rest will follow.

Routing isn't something that you need to memorize, per se. It's just something that you need to understand once.

And once you really understand it, every back-end system is going to start to look familiar.